

```

module parcialB_tb;
    logic clk, reset, load, pout;
    logic T3:03 init, q;

    parcial dut (.clk(clk), .reset(reset), .load(load),
        .pout(pout), .init(init) ,.q(q));

    always begin clk = ~clk; #5
    initial begin
        clk = 0; reset = 1; load = 0; init = 4'b0;
        #0; reset = 0; #1; init = 4'b0010;
        load = 1; #10; load = 0; #20; init = 4'b1100;
        #10; load = 1; #10;
        $display("Valor final de q: %.b", q);
        $stop;
    end
end module

```

Ejercicio 2)

Se requiere analizar las modificaciones necesarias al procesador LEGv8 de un ciclo con manejo de EXCEPCIONES (Práctico 3) a fin de separar los vectores de excepciones de sistema (op code invalido para nuestro caso), del vector de interrupciones por periféricos (ExtIRQ). Los nuevos vectores son:

- OpCode Invalido = 0x00D8
- Interrupción de E/S externa (ExtIRQ) = 0x00F8

NOTA: La señal de respuesta ExcAck solo debe ponerse en "1" en caso de ejecutarse la ISR de una Interrupción de E/S externa. Caso contrario debe permanecer en "0".

Proponga una modificación al procesador y responda:

a) Es necesario agregar señales de control extras?

NO.....

Si: ☒ Nombre

#Origen

#Destino

Exctype

ESR (Floppy)

Fetch (mux)

b) Enumere qué módulos existentes deben ser modificados (si los hubiera)

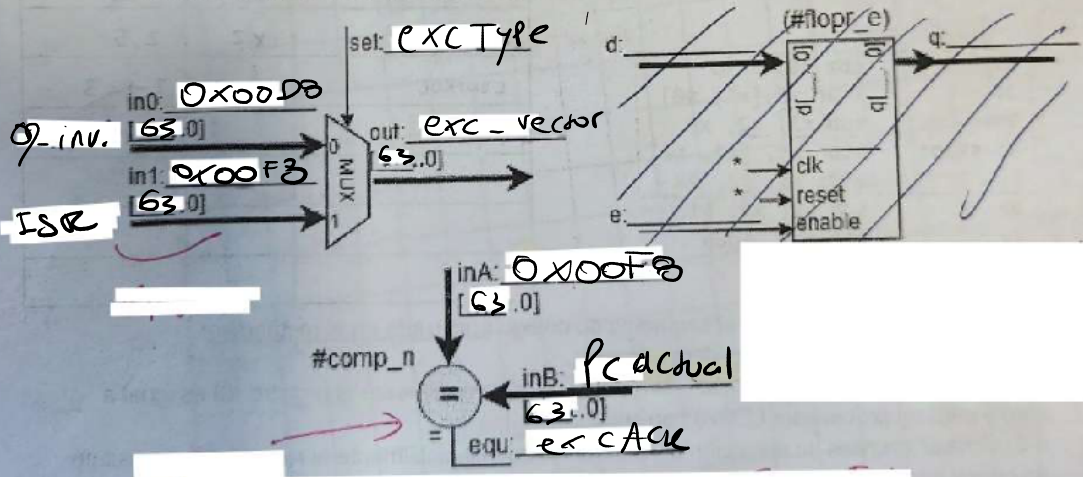
1 Mux del fetch

2

3

4

c) Se requiere agregar alguno de éstos módulos? De ser afirmativo, especificar el nombre de las señales involucradas. Caso contrario, tachar el/los módulos.



d) Suponga que el contenido de la dirección de memoria 0x00FC no contiene una instrucción válida. Que ocurre en nuestro nuevo procesador si ocurre una interrupción de E/S externa? (marcar la correcta)

- ☐ La ISR de interrupción de E/S externa se ejecuta normalmente
- ☐ El procesador se queda en un bucle infinito sobre el vector 0x00F8
- ☐ El procesador se queda en un bucle infinito sobre el vector 0x00D8
- ☒ Ninguna de las anteriores

Ejercicio 3

Asumiendo que las etapas individuales del pipeline de un procesador tienen las siguientes latencias:

	IF	ID	EX	WB
Latencia por etapa	100 ns	80 ns	130 ns	105 ns

a) Completar:

Tipo de procesador	Máxima frecuencia de reloj [Hz]	Latencia de una instrucción [ns]
Sin pipeline	2,4 MHz	415
Con pipeline	7,69 MHz	520

Asuma que quiero ejecutar un fragmento de código que tiene 2E6 instrucciones, las cuales no presentan hazards: $\rightarrow 2000000 (2 \times 10^6)$

- b) ¿Cuánto tiempo llevaría ejecutar dicho fragmento de código? $520 + (1.999.999 \times 130) = 0,26s$
- c) ¿Cuánto tiempo llevaría ejecutar el mismo fragmento si el procesador no tuviese pipeline? $415 \times 2 \times 10^6 = 0,83s$

Tipo de dependencia	Registro	Instrucciones
---------------------	----------	---------------

Ejercicio 4

Para la siguiente secuencia de código:

loop:

```

1>      cbz x0, skip
2>      ldur x2, [x1, #0]
3>      sub x3, x2, x0
4> skip: stur x3, [x1, #0]
5>      addi x1, x2, #8
6>      sub x9, x9, #1
7>      cbnz x9, loop
    
```

Tipo de dependencia	Registro	Instrucciones
Control	-	1
Datos (Raw)	x2	2-3
Datos - cond	x2	2-5
Datos - cond	x3	3-4

a) Considerando únicamente el segmento de código encerrado en el rectángulo:

- a-1) Completar la tabla con las dependencias.
- a-2) Mostrar el orden de ejecución considerando que inicialmente el registro X0 es igual a cero y para un procesador LEGv8 con stall.
- a-3) Mostrar el orden de ejecución considerando que el inicialmente el registro X0 es distinto de cero y para un procesador LEGv8 con forwarding stall.
- ... inicialmente el registro x9 está inicializado en

$X_0 = 0$; Stall

I	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
CBZ(1)	IF	ID	EX	Mem	Wb					
LDUR(2)		IF	ID	EX	⊗	⊗				
...			IF	ID	⊗	⊗	⊗			
...				IF	⊗	⊗	⊗	⊗		
STUR(4)					IF	ID	EX	Mem	Wb	
ADDI(5)						IF	ID	EX	Mem	Wb

⊗ Flush

$X_0 \neq 0$; Forwarding stall

I	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
CBZ(1)	IF	ID	EX	Mem	Wb					
LDUR(2)		IF	ID	EX	Mem	Wb				
SUB(3)			IF	ID	⊗	⊗	⊗			
SUB(3)				IF	ID	EX	Mem	Wb		
STUR(4)					IF	ID	EX	Mem	Wb	
ADDI(5)						IF	ID	EX	Mem	Wb

⊗ Stall

de cero y para un procesador LEGv8 con forwarding stall.

- b) Considerando el código completo, que inicialmente el registro x9 está inicializado en 1000, x0 es distinto de cero y para un procesador LEGv8 con forwarding stall:
- b-1) ¿Cuántos ciclos de clock tomaría ejecutar este código? 12997
- b-2) ¿Cuántos ciclos de clock tomaría ejecutar este código en el microprocesador optimizado para saltos: 10999
- b-3) Calcular la ganancia de velocidad entre los puntos b-1 y b-2: 1,182
- Respuestas puntos a-2 y a-3 en la siguiente hoja

```

loop:
1>      cbz x0, skip
2>      ldur x2, [x1, #0]
3>      sub x3, x2, x0
4> skip: stur x3, [x1, #0]
5>      addi x1, x2, #8
6>      sub x9, x9, #1
7>      cbnz x9, loop
  
```

$X_0 \neq 0 \Rightarrow 10$ ciclos

Se repite 1000 veces.

Como cada salto no tomado ^{erróneamente} tiene penalización de 3 ciclos $\Rightarrow$ tengo 3×999 ciclos de más.
(la última iteración se toma el salto correctamente)

$$\begin{aligned}\text{Clocks totales} &= 10 \times 1000 + 3 \times 999 \\ &= 12.997\end{aligned}$$

Proce optimizado para saltos: 1 ciclo de penalización

$$\begin{aligned}\text{Clocks totales} &= 10 \times 1000 + 999 \\ &= 10.999\end{aligned}$$

$$\text{Garancia en clocks: } \frac{12.997}{10.999} = 1,182$$